

DTO, Mapper, Pagination & Specification

Dr. Öğr. Üyesi Alper Talha KARADENİZ

DTO Nedir?

- DTO (Data Transfer Object - Veri Transfer Nesnesi), iki uygulama arasında aktarılan verileri modellemek için kullanılan nesnelere verilen isimdir.

Temel Özellikleri ve Kullanım Alanları

- **Veri Taşıma Sorumluluğu:** DTO'ların temel sorumluluğu, bir istemci (client) ile bir sunucu (server) arasında veya iki farklı backend servisi arasındaki veri alışverişini temsil etmektir.
- **REST Servisleri ile Kullanım:** Özellikle RESTful web servislerinde, HTTP isteğinin gövdesini (request body) veya sunucudan dönen yanıt gövdesini (response body) modellemek için yaygın olarak tercih edilirler.

Temel Özellikleri ve Kullanım Alanları

- **Basit Yapı:** DTO'lar genellikle herhangi bir iş mantığı (business logic) içermeyen, sadece veriyi tutan özniteliklerden (attributes) ve bunlara erişim sağlayan metotlardan oluşan basit Java nesneleridir (POJO).
- **Örnek Senaryo:** Bir para transferi uygulamasında, istemcinin gönderdiği gönderici ID'si, alıcı ID'si ve miktar gibi bilgileri içeren TransferRequest sınıfı bir DTO örneğidir,. Sunucu bu nesneyi alır, içindeki veriyi işler ve gerekirse yanıt olarak başka bir DTO döner.

Entity neden direkt Controller'da kullanılmaz?

- **Sorumlulukların Ayrılması ve Bağlantısızlık (Decoupling):** Uygulama tasarımında bileşenlerin birbirinden ayrılması, bakımın kolaylaştırılması ve genişletilebilirlik açısından kritik bir en iyi uygulamadır. Entity sınıfları veritabanı tablosuyla eşleşen ve veri kalıcılığından sorumlu yapılardır. DTO'lar ise sadece iki uygulama arasında taşınacak veriyi modellemekten sorumludur. Bu ayrım, veritabanı yapısı değiştiğinde dış dünyaya sunulan API'nin bozulmamasını sağlar.

Entity neden direkt Controller'da kullanılmaz?

- **Güvenlik ve Veri Gizliliği:** Entity sınıfları genellikle veritabanı kimlikleri (ID), şifreler veya iç işlemler için kullanılan hassas veriler içerir. DTO kullanımı, istemciye sadece görmesi gereken verilerin gönderilmesini ve istemciden sadece gerekli verilerin alınmasını sağlayarak bir güvenlik katmanı oluşturur.
- **API Esnekliği ve Karmaşıklığı Önleme:** Bir Entity, veritabanı gereksinimleri nedeniyle karmaşık ilişkilere (@OneToMany, @ManyToMany vb.) sahip olabilir. Bu karmaşık nesneleri doğrudan Controller üzerinden dışarıya açmak, gereksiz veri trafiğine ve performans kayıplarına yol açabilir. DTO'lar, istemcinin ihtiyacına göre veriyi düzleştirilmiş (flat) ve basit bir Java nesnesi (POJO) olarak sunar.

Entity neden direkt Controller'da kullanılmaz?

- **İstenmeyen Veri Kayıplarını Önleme:** Örneğin, bir HTTP PUT isteğinde doğrudan bir Entity nesnesi kullanıldığında, istemci tüm alanları göndermezse, gönderilmeyen alanlar veritabanında yanlışlıkla null olarak üzerine yazılabilir. DTO'lar, istemciden gelen veriyi filtreleyerek bu tür hatalı güncellemelerin önüne geçer.
- **Doğrulama ve İş Mantığı:** Giriş verilerini doğrulamak için kullanılan kurallar (örneğin @NotNull, @Size) genellikle Entity seviyesinde değil, Controller seviyesindeki giriş nesnelerinde (DTO) tanımlanmalıdır. Bu, veritabanı modeli ile giriş doğrulama mantığını birbirinden ayırır.

Soyutlamanın (Abstraction) Sebepleri

- **Bağımlılıkların Azaltılması (Decoupling):** Uygulama bileşenleri arasında doğrudan bağ (direct coupling) kurulması engellenir. Bu sayede, örneğin bir bildirim servisi e-posta yerine başka bir kanalla gönderilmeye başlandığında, bu servisi kullanan ana iş mantığı kodunun değiştirilmesi gerekmez.
- **Değişime Uyum Sağlama:** Yazılım geliştirme sürecinde "değişim" tek sabittir. Soyutlamalar (arayüzler), bir veritabanı teknolojisinden (örneğin ilişkiselden NoSQL'e) diğerine geçiş yaparken veya harici servisler güncellendiğinde kodun etkilenmesini en aza indirir.

Soyutlamanın (Abstraction) Sebepleri

- **Karmaşıklığı Yönetme:** Spring'in IoC (Inversion of Control) konteyneri, nesne oluşturma ve nesneler arası ilişkileri kurma sorumluluğunu geliştiriciden alır. Bu soyutlama katmanı sayesinde geliştiriciler, düşük seviyeli yapılandırma detayları yerine doğrudan kullanıcı beklentilerini karşılayan iş mantığına (business logic) odaklanabilir.

Güvenliğin (Security) Sebepleri

- **Yetkisiz Erişimi Engelleme:** Bir REST API veya Actuator gibi iç gözlem uç noktaları dünyaya açık olduğunda, kimlik doğrulaması (authentication) yoksa kötü niyetli kişiler verileri ekleyebilir, silebilir veya uygulamanın iç durumunu görebilir.
- **Veri ve İşlem Bütünlüğü:** CSRF (Cross-Site Request Forgery) gibi güvenlik önlemleri, özellikle üretim (production) ortamındaki uygulamalarda formların ve işlemlerin manipüle edilmesini önlemek için hayati önem taşır.

Güvenliğin (Security) Sebepleri

- **Merkezi Denetim:** Güvenlik ve günlükleme (logging) gibi "kesişen ilgiler" (cross-cutting concerns), ana iş mantığından ayrılarak merkezi bir noktadan yönetilir. Spring Security, kullanıcıları doğrulamak ve yetkisiz erişimi önlemek için bu süreci standartlaştırır.

RequestDTO vs ResponseDTO

Özellik	RequestDTO	ResponseDTO
Yön	İstemci → Sunucu	Sunucu → İstemci
Yaygın Anotasyon	@RequestBody	@ResponseBody veya @RestController
Kritik Rolü	Giriş doğrulaması (Validation) ve veri alımı.	Veri gizleme (şifre vb.) ve API esnekliği [Conversation History].
Örnek Yapı	RegistrationForm, TransferRequest.	Country, ErrorDetails, PaymentDetails.

Mapping nedir?

- Spring ekosisteminde mapping (eşleme) kavramı, temel olarak farklı veri formatları, protokoller veya uygulama katmanları arasında bağlantı kurma ve veri dönüştürme işlemini ifade eder.

İstek Eşleme (Request Mapping)

- Spring MVC ve WebFlux çerçevelerinde mapping, bir HTTP isteğinin (belirli bir URL yolu ve HTTP metodu) arka uçtaki belirli bir denetleyici (controller) metoduna bağlanması sürecidir.
- **Anotasyonlar:** `@RequestMapping`, `@GetMapping` veya `@PostMapping` gibi anotasyonlar, hangi ağ isteğinin hangi Java kodunu tetikleyeceğini tanımlar.
- **Handler Mapping:** Spring'in iç mekanizmasında yer alan Handler Mapping bileşeni, her gelen istek için uygun olan eşleşmiş metodu bulmaktan sorumludur.

Varlık ve Tablo Eşleme (ORM - Object-Relational Mapping)

- Veritabanı işlemlerinde mapping, Java sınıflarının (entities) ilişkisel veritabanındaki tablolarla ve sınıf alanlarının sütunlarla eşleştirilmesidir.
- **Uygulama:** @Entity anotasyonu bir sınıfı veritabanı tablosuyla eşlerken; @Id alanı birincil anahtarla, @Table ise sınıfın hangi isimdeki tabloya yazılacağıyla eşleştirir. Bu sayede geliştirici, SQL yazmak yerine doğrudan nesneler üzerinden veritabanı işlemi yapabilir.

Veri Dönüştürme ve Format Eşleme

- Farklı veri yapıları arasındaki geçişler de mapping olarak adlandırılır:
- **RowMapper:** JDBC ile çalışırken, veritabanından dönen ham satır verilerinin (ResultSet) Java nesnelere aktarılması işlemini RowMapper bileşeni üstlenir.
- **JSON/Object Eşleme:** REST API'lerde, gelen JSON formatındaki verilerin Java nesnelere (deserialization) veya nesnelerin JSON'a (serialization) dönüştürülmesi bir tür veri eşlemedir. Bu işlem genellikle arka planda Jackson kütüphanesi tarafından otomatik olarak yapılır.

Manuel Mapping

- Spring ekosisteminde manuel mapping (el ile eşleme), genellikle verilerin farklı katmanlar veya formatlar arasında (örneğin veritabanından Java nesnesine veya Entity'den DTO'ya) geliştirici tarafından yazılan özel kodlar aracılığıyla dönüştürülmesini ifade eder.

JdbcTemplate ve RowMapper ile Veri Eşleme

- ilişkisel veritabanlarıyla çalışırken JdbcTemplate kullanıldığında, veritabanından dönen ham satırların (ResultSet) Java nesnelere dönüştürülmesi için sıklıkla manuel mapping uygulanır.
- **RowMapper Kullanımı:** Geliştirici, RowMapper arayüzünü uygulayarak her bir sütunun nesnenin hangi alanına karşılık geleceğini açıkça belirtir.
- **Örnek:** Bir AccountRowMapper sınıfı içinde resultSet.getInt("id") gibi ifadelerle veritabanı sütunları manuel olarak Java sınıfının özniteliklerine atanır.
- **Neden Tercih Edilir?:** JdbcTemplate, karmaşık nesne-ilişkisel eşleme (ORM) çerçevelerine ihtiyaç duyulmayan küçük projelerde veya tam kontrol istenen durumlarda bu manuel yöntemi kullanır.

DTO ve Entity Arasındaki Dönüştürme

- Uygulama tasarımında Entity sınıflarının (veritabanı modelleri) doğrudan dış dünyaya açılmaması gerektiği durumlarda, veriler DTO (Data Transfer Object) sınıflarına manuel olarak aktarılır.
- **İşleyiş:** Bir denetleyici (controller) veya servis katmanında, gelen bir RequestDTO nesnesindeki veriler alınır ve manuel olarak bir Entity nesnesinin alanlarına set edilir.
- **Güvenlik ve Esneklik:** Bu manuel ayırım, istemciye sadece görmesi gereken verilerin gönderilmesini ve veritabanı yapısındaki değişikliklerin API'yi hemen bozmamasını sağlar.

JSON ve Tip Eşleme (Jackson)

- Mesajlaşma sistemlerinde (JMS gibi) veya REST API'lerde, verilerin JSON'dan nesneye dönüşümü sırasında bazen manuel müdahale gerekebilir.
- **Tip Kimlik Eşlemesi:** MappingJackson2MessageConverter kullanılırken, Java sınıfları ile mesaj başlıklarındaki tip kimlikleri (type ID) arasındaki bağlantı `setTypeIdMappings()` metodu ile manuel olarak kurulabilir. Bu, gönderici ve alıcının farklı paket yapılarında olmasına rağmen aynı veriyi tanınmasını sağlar.

İstek Eşleme (Request Mapping)

- Daha basit bir düzeyde mapping, bir HTTP isteğinin (URL ve Metot) arka uçtaki bir Java metoduna bağlanmasıdır.
- **Anotasyonlar:** `@RequestMapping`, `@GetMapping` veya `@PostMapping` gibi anotasyonlar kullanılarak, hangi ağ yolunun hangi kodu tetikleyeceği geliştirici tarafından manuel olarak tanımlanır.

Mapper class

- Spring ekosisteminde, özellikle veritabanı ve veri transferi işlemleri sırasında, bir "Mapper" sınıfı (veya arayüzü), verilerin bir formattan diğerine (örneğin veritabanı satırından Java nesnesine) dönüştürülmesi sorumluluğunu üstlenir. Kaynaklarda bu bağlamda en çok öne çıkan yapı RowMapper'dır.

RowMapper (JDBC Veri Eşleme)

- RowMapper, Spring JDBC'nin temel bir parçasıdır ve bir veritabanı sorgusundan dönen her bir satırı (ResultSet) belirli bir Java nesnesine dönüştürmek için kullanılır.
- **Çalışma Mantığı:** JdbcTemplate bir sorgu çalıştırdığında, sonuç kümesindeki her bir satır için RowMapper arayüzündeki mapRow() metodunu çağırır.
- **mapRow() Metodu:** Bu metot, o anki satırı (ResultSet) ve satır numarasını parametre olarak alır; geliştirici bu metot içinde veritabanı sütunlarını Java nesnesinin özniteliklerine manuel olarak atar.
- **Uygulama:** Küçük projelerde veya tam kontrol istenen durumlarda, bir Ingredient veya Purchase nesnesi oluşturmak için bu manuel eşleme yöntemi tercih edilir.

Modern Kullanım Yöntemleri

- Geliştiriciler, her dönüşüm için açıkça bir RowMapper sınıfı yazmak yerine şu modern yaklaşımları kullanabilirler:
- **Lambda İfadeleri:** Dönüşüm mantığı doğrudan JdbcTemplate metodunun içinde anonim bir fonksiyon olarak tanımlanabilir.
- **Metot Referansları:** Java'nın metot referansı özelliği (örneğin `this::mapRowToIngredient`) kullanılarak kod daha okunabilir ve sade hale getirilebilir.

Mesaj ve Format Dönüştürücü Mapperlar

- REST servisleri ve mesajlaşma sistemlerinde (JMS, RabbitMQ) verilerin format değiştirmesi de "mapping" olarak adlandırılır:
- **MappingJackson2MessageConverter:** Nesneleri JSON formatına (veya tersine) dönüştürmek için Jackson kütüphanesini kullanan bir dönüştürücüdür.
- **JmsHeaderMapper:** JMS başlıklarını standart mesaj başlıklarıyla eşlemek (mapping) için kullanılır.

Pagination (Sayfalama) Nedir?

- Bir uygulamada binlerce kayıt olduğunda, tüm bu verileri tek bir seferde yüklemek hem performansı düşürür hem de kullanıcı için karmaşaya neden olur. Sayfalama, bu verileri belirli bir sayfa numarası ve sayfa boyutu (bir sayfada kaç kayıt olacağı) temelinde alt kümelere ayırarak çekme işlemidir.

Pagination (Sayfalama) Nedir?

- **Pageable ve PageRequest:** Spring Data'da sayfalama isteği oluşturmak için Pageable arayüzü ve bunun bir uygulaması olan PageRequest sınıfı kullanılır. Örneğin, PageRequest.of(0, 20) ifadesi, ilk sayfayı (0. indeks) ve sayfa başına 20 kaydı temsil eder.
- **Dönüş Tipi:** Sayfalanmış bir sorgu genellikle sadece verileri değil, toplam sayfa sayısı gibi meta verileri de içeren bir Page nesnesi döndürür.

Sorting (Sıralama) Nedir?

- **Sort Sınıfı:** Spring Data'da sıralama kriterlerini tanımlamak için Sort sınıfı kullanılır. Örneğin, `Sort.by("createdAt").descending()` ifadesi, kayıtları oluşturulma tarihine göre en yeniden en eskiye doğru sıralar.
- **Metot İsimlerinden Sıralama:** Repozitori metot isimlerine `OrderBy...Desc` veya `OrderBy...Asc` eklenerek (örneğin `findByUserOrderByPlacedAtDesc`) otomatik sıralama sorguları üretilebilir.

Spring Data ve REST API'lerde Kullanımı

- **PagingAndSortingRepository:** Spring Data, temel CRUD işlemlerine ek olarak sayfalama ve sıralama yetenekleri sunan PagingAndSortingRepository arayüzünü sağlar. Bu arayüz, sonuçları sayfa sayfa almanıza ve sıralamanıza olanak tanıyan ek metotlar içerir.
- **Spring Data REST:** Eğer Spring Data REST kullanılıyorsa, otomatik oluşturulan uç noktalar (endpoints) sorgu parametrelerini doğrudan destekler. Örneğin, bir sorgu şu şekilde yapılabilir:
localhost:8080/api/tacos?sort=createdAt,desc&page=0&size=12. Bu istek, oluşturulma tarihine göre sıralanmış ilk sayfadaki 12 kaydı getirir.

Pageable

- Spring ekosisteminde Pageable, Spring Data tarafından sunulan ve veritabanı sorgu sonuçlarını belirli bir sayfa numarası ve sayfa boyutu (bir sayfadaki kayıt sayısı) temelinde alt kümelere ayırarak çekmek için kullanılan bir arayüzdür.

Page<T>

-
- Spring ekosisteminde Page<T>, Spring Data tarafından sunulan ve veritabanı sorgu sonuçlarının bir alt kümesini (dilimini) bu dilime ait meta verilerle birlikte tutan özel bir konteyner nesnesidir.

Sort

- Spring ekosisteminde Sort (Sıralama), veritabanı sorgularından dönen sonuçların belirli bir mülke (property) göre belirli bir düzende (artan veya azalan) getirilmesini sağlayan bir mekanizmadır.

Metot İsimlerinden Sorgu Üretme (Query Derivation)

- Spring Data JPA'nın en temel dinamik özelliklerinden biridir. Geliştirici, repositori arayüzünde belirli bir isimlendirme kuralına göre metot tanımlar ve Spring Data bu metot ismini analiz ederek otomatik olarak SQL/JPQL sorgusu üretir.

Metot İsimlerinden Sorgu Üretme (Query Derivation)

- **İşleyiş:** Bir metot ismi; fiil (find, read, get), isteğe bağlı bir özne, "By" kelimesi ve bir yüklemden (predicate) oluşur.
- **Esneklik:** Sorgu, metoda geçilen parametrelere göre şekillenir. Örneğin, `findByDeliveryZip(String zip)` metodu, verilen ZIP koduna göre dinamik bir filtreleme yapar.
- **Operatörler:** Sorgular; `Between`, `IsNotNull`, `GreaterThan`, `Like`, `IgnoringCase` gibi operatörlerle parametre değerlerine göre dinamik olarak zenginleştirilebilir.

Dinamik Filtreleme ve Arama Senaryoları

- **Kombinasyonlar:** Bir kullanıcı ürünleri ismine, fiyatına veya markasına göre aramak isteyebilir. Herhangi bir kombinasyon (sadece isim, hem isim hem marka vb.) mümkün olduğunda, backend bu değişken talepleri karşılayacak bir yapı sunmalıdır.
- **İsteğe Bağlı Parametreler:** `@RequestParam(required = false)` gibi tanımlamalarla, istemcinin göndermediği parametreler sorgu mantığına dahil edilmeyebilir, bu da sorgu sonucunun dinamik olarak değişmesini sağlar.

Dinamik Sıralama ve Sayfalama (Pagination & Sorting)

- **Sort ve Pageable:** Sort nesnesi kullanılarak verilerin hangi alana göre (örneğin createdAt) ve hangi yönde (ASC/DESC) sıralanacağı çalışma zamanında kararlaştırılabilir.
- **URL Üzerinden Kontrol:** Spring Data REST gibi araçlar sayesinde istemci, URL üzerinden ?sort=name,desc&page=0&size=10 gibi parametrelerle sorgunun davranışını dinamik olarak değiştirebilir.

@Query Anotasyonu

- Metot isimlendirme kurallarının yetersiz kaldığı çok karmaşık veya özel durumlarda, @Query anotasyonu ile belirli bir mantık çerçevesinde sorgular yazılabilir. Her ne kadar @Query içindeki SQL/JPQL ifadesi statik görünse de, kullanılan parametreler (:name gibi) sorgunun ürettiği sonucun dinamik olmasını sağlar.